

Sensiq: An MCU-Based IoT Architecture for Environmental Monitoring

Technical Reports: CL-2026-42, July 2026

Simon Völkl, Johannes Schieder, Sebastian Rosner and Andre Tien Vu 

Department of Electrical Engineering, Media, and Computer Science

Ostbayerische Technische Hochschule Amberg-Weiden

Amberg, Germany

Abstract—Background: Reliable environmental monitoring is essential for laboratory and facility safety, yet many systems trade off scalability, security, or real-time access. **Objective:** This concept paper proposes Sensiq, an MCU-based IoT architecture intended to enable secure, low-latency live monitoring and scalable historical storage for environmental sensor data.

Methods: We describe a serverless AWS-based pipeline in which ESP32 edge devices publish JSON-encoded measurements (temperature, humidity, gas, flame) over MQTT/TLS. The architecture specifies Lambda-based validation and persistence to DynamoDB and S3, and REST endpoints for ‘GET /live’ and ‘GET /history’. We propose a configurable simulator to exercise ingestion and retrieval without physical hardware and to guide future implementation and testing.

Conclusion: Sensiq defines a lightweight, extensible foundation for laboratory environmental monitoring. Future work focuses on implementing the MVP, adding a web dashboard and notification services for visualization and alerts.

Index Terms—IoT, Environmental Monitoring, ESP32, MCU, MQTT, Serverless Architecture, AWS

I. INTRODUCTION

A. Motivation

In modern laboratory and production environments, maintaining stable ambient conditions is vital for experimental validity and product quality. Conventional monitoring systems often lack real-time accessibility, scalability, or automated alerting capabilities. This project addresses these gaps by integrating cost-effective ESP32 edge hardware with a robust AWS cloud architecture. By transitioning from isolated local measurements to a centralized, serverless IoT platform, we enable seamless data persistence, interactive visualization, and immediate incident response. The objective is to provide lab managers and technicians with a scalable, high-availability tool that ensures operational safety and data-driven quality assurance.

This Section introduces the motivation, the Unique Selling Point (USP), and the key technical components of the proposed system. Section II gives an overview of related work and technologies that inspired and influenced the design of our system. The core requirements are defined in Section III with a focus on user-centered design, including detailed user stories and acceptance criteria for the minimum viable product (MVP). The technical concept and architecture of the system are described in Section IV. Section V summarizes the key findings and outlines future work and next steps for the project.

B. Unique Selling Point

The proposed system is specifically engineered to meet the rigorous demands of high-precision laboratory monitoring, such as maintaining stable environmental conditions for sensitive chemical samples. The architecture differentiates itself through several key strategic factors:

- **Cost-Effective Custom Infrastructure:** A significant drawback of many existing solutions is the heavy reliance on third-party dashboarding tools like Grafana, which often incur substantial licensing costs and complex login management in multi-user environments. Our system eliminates these barriers by utilizing a custom frontend, providing a tailored, high-performance interface without per-user licensing fees.
- **Advanced Data Governance:** By implementing a fully serverless data pipeline with the system ensures long-term data sovereignty and cost-efficiency. Effectively storing data enables laboratory managers to perform high-speed safety audits and complex trend analysis over years of data with minimal operational overhead.
- **High-Performance Architecture:** Moving beyond simple academic prototypes, this project leverages a MCU with good processing power and integrated security features. Combined with the robustness of serverless components in the cloud, the system ensures sub-second response times for critical events, such as gas leaks or temperature spikes.

C. Key Technical Components

The proposed system is built around an ESP32 edge device connected to multiple environmental sensors, such as temperature, humidity, gas, and flame sensors. Sensor readings are pre-processed locally and transmitted via encrypted MQTT/TLS to protect data confidentiality and ensure device authentication during communication.

AWS IoT Core serves as the secure entry point between the devices and the cloud backend. It receives incoming measurements and makes them available for further processing of other AWS services. Live sensor values are handled by a dedicated processing pipeline and exposed through a REST interface for external access, while historical measurements are stored and made available through a separate REST endpoint for time-series retrieval and analysis.

Future extensions may include a frontend dashboard that provides user-oriented visualization and analysis tools for monitoring trends and comparing sensor values over time. In addition, AWS SNS can be integrated later to send email notifications to users when critical thresholds are exceeded.

Challenges could include handling large volumes of data, ensuring low-latency processing and communication, and securing user data from unauthorized access.

II. RELATED WORK

A. IoT Applications in Laboratory Safety

The integration of Internet of Things (IoT) solutions into safety-critical environments has become a focal point of recent research. A highly relevant study by Ramadhani, Aini, and Jimmie explores the development of an air quality monitoring system specifically for hazardous gas detection in chemical laboratories [1]. Their architecture utilizes a NodeMCU ESP8266 microcontroller paired with MQ-135 gas sensors and DHT11 temperature sensors to mitigate risks such as poisoning and explosions.

While their research confirms that low-cost sensor nodes are effective as early warning tools, their implementation relies on the Blynk platform for data visualization. This introduces dependencies on external service providers and limits the ability to integrate custom business logic, such as the multi-persona alerting system (for lab technicians and production managers) implemented in this project. In contrast, our system is planned to implement a dedicated AWS-native pipeline to ensure full data sovereignty and professional-grade scalability.

B. Performance and Scalability of Serverless Architectures

Beyond the hardware implementation, the efficiency of the underlying cloud infrastructure is crucial for real-time safety monitoring. Gudduri investigated the performance of "lean" serverless architectures for processing IoT data streams [2]. By replaying large-scale datasets through a pipeline consisting of Amazon Kinesis, AWS Lambda, and Amazon S3, the study proved that sub-second response times are achievable in modern cloud environments.

The findings of Gudduri show a median end-to-end latency of 163 ms, even at high throughput levels, by utilizing techniques such as provisioned concurrency to mitigate "cold-start" delays. These technical benchmarks provide the empirical justification for the serverless architecture chosen in this project, ensuring that safety-relevant gas alerts are processed within the timeframes required for emergency response.

C. Comparison of Existing Solutions

Table I summarizes the positioning of the proposed system in relation to the discussed academic works and common commercial alternatives.

III. REQUIREMENTS

The requirements for the final system vision are defined in the form of persona-based user stories including acceptance criteria, prioritization using the MoSCoW method and the definition of the minimum viable product (MVP).

Table I
COMPARATIVE ANALYSIS OF MONITORING ARCHITECTURES

Feature	Ramadhani (2026)	Gudduri (2025)	Sensiq (Ours)
Real-Time Alerts	Yes	Yes	Yes
Long-Term Audits	Limited	High	High
Custom API/UI	No	No	Yes
Cost Efficiency	High	High	High
Lab Focus	Yes	No	Yes
Suitability	Medium	Medium	High

A. Project Personas

To ensure the system meets the diverse needs of all stakeholders, the requirements are guided by the following personas. Each persona has a specific role, defined goals, and distinct interests in the system:

Chemist Christian Focuses on exact environmental conditions. His primary goal is to monitor precise laboratory temperatures to accurately run experiments and evaluate environment-dependent reactions.

Occupational Safety Officer Olivia Prioritizes employee well-being. She requires immediate, clear indicators of air quality safety to proactively enforce protective measures and prevent workplace health hazards.

Facility Manager Martin Responsible for building operations and rapid incident response. He needs instant, unmissable alerts regarding critical sensor thresholds to take immediate corrective actions before physical risks escalate.

Project Manager Pia Looks at the big picture and long-term optimization. She aims to analyze historical data trends to identify recurring patterns, correlate them with facility operations, and implement targeted strategies for improving environmental conditions.

IT Administrator Ian Is responsible for the technical infrastructure and stability of the sensor network. He needs to check the connection status and last heartbeat of each sensor at any time to detect offline devices early and resolve network issues before data gaps arise.

Data Scientist Daniela Works with historical sensor data to develop predictive models for air quality. She requires flexible access to raw data with adjustable time ranges and resolutions to build reliable training datasets.

B. User Stories

1) Real-time Temperature Monitoring:

As Chemist Christian, I want to see the exact current temperature in the laboratory, so that I can accurately plan my experiments and evaluate temperature-dependent reaction rates.

Acceptance Criteria:

- The dashboard displays the current temperature precisely in degrees Celsius. Input: latest DB temperature readings. Output: exact numerical temperature value.

- The temperature value updates automatically when a new reading arrives — no manual reload required. Test: write new temperature values to DB; display must update within 10 s.
- Displayed temperature values are at most 30 s old. Test: compare value timestamps with system time; difference ≤ 30 s.
- If no current temperature data is available (gap > 30 s), a notice about missing data is shown instead.

2) Workplace Air Quality Status:

As Occupational Safety Officer Olivia, I want to see at a glance whether the air quality in the workspace is currently safe or critical, so that I can immediately initiate protective measures for the employees if necessary.

Acceptance Criteria:

- The dashboard shows the air quality as a color-coded indicator based on safety thresholds: Green (safe) and Red (critical). Input: latest DB air quality readings. Output: correct status color.
- The status color updates automatically when a new reading arrives — no manual reload required. Test: write new air quality values to DB; display must update within 5 s.
- The displayed status is based on data that is at most 30 s old. Test: compare value timestamps with system time; difference ≤ 30 s.

3) Critical Alert:

As a facility manager Martin, I want an immediate, unmissable notification the second a sensor reading hits a critical limit, empowering me to take instant corrective action before the situation escalates into a physical health risk.

Acceptance Criteria:

- An email notification is sent automatically within 10 s of a threshold exceedance.
- The notification contains: measured value (incl. unit), measurement timestamp, and alert message type.
- Consecutive exceedances of the same type within 5 min do not trigger a further email (debounce).
- The notification is delivered even if the frontend is not open at trigger time.

4) 24-Hour History Chart:

As Project Manager Pia, I want to analyze the sensor data from the past 24 hours in a clear historical chart, so that I can identify recurring patterns of poor air quality, correlate them with specific facility operations, and implement targeted ventilation strategies.

Acceptance Criteria:

- Dashboard displays a line chart with readings from the last 24 hours.
- Chart is fully interactive within 3 s of page load.

- Periods without data are shown as visible line breaks — no silent interpolation.
- Chart updates automatically when new readings arrive, without a page reload.

5) Custom History View:

As Data Scientist Daniela, I want to extract historical sensor readings with a customizable time range and sampling resolution, so that I can generate precise, high-granularity datasets to train our predictive air-quality models.

Acceptance Criteria:

- Dashboard provides date pickers for start/end and a resolution selector (hour/day).
- A query for 30 days at daily resolution returns results in under 5 s.
- If an end date before the start date is entered, a clear error message is shown and no query is executed.
- Periods without data appear as an explicit empty row in the table and as a line break in the chart.

6) Sensor Status:

As IT Administrator Ian, I want to instantly verify the connection status and the last heartbeat timestamp of each sensor, so that I can proactively detect offline devices and troubleshoot network dropouts before data gaps occur.

Acceptance Criteria:

- Sensor status Active/Inactive is prominently visible. Status switches to Inactive when the last data point is more than 2 min ago.
- Date and time of the last received reading are displayed next to the status (format: DD.MM.YYYY HH:MM:SS).
- Status indicator updates automatically — no manual reload required.
- If the database connection is temporarily unavailable, a notice is shown instead of the status.

C. Definition of Done

A user story is considered complete only when, in addition to its specific acceptance criteria, all of the following quality standards are met:

- Code Review: Reviewed and approved by at least one other developer (four-eyes principle).
- Tests: Unit tests written and passing; coverage $\geq 40\%$.
- Documentation: Technical docs and API interfaces updated where applicable.
- Deployment: Feature successfully deployed.
- Functional Test: Feature manually tested in staging without issues.

D. Prioritisation (MoSCoW)

The user stories are prioritized using the MoSCoW method, which categorizes requirements into Must Have, Should Have, Could Have, and Won't Have, as shown in Table II.

Table II
MoSCoW PRIORITIZATION OF USER STORIES

Category	User Stories
Must Have	Current Status Display, Critical Alert, Workplace Air Quality
Should Have	Sensor Status
Could Have	24-Hour History Chart
Won't Have	Custom History View

E. Minimum Viable Product (MVP)

The MVP implements a server-side IoT pipeline on Amazon Web Services (AWS). It focuses on ingesting simulated sensor data via MQTT, processing and storing it within the AWS ecosystem, and providing access through a Web API. This serves as the technical foundation for the system.

Instead of physical hardware, a simple script simulates realistic sensor data. Messages are transmitted via the MQTT protocol to AWS IoT Core, ensuring a lightweight and software-defined testing environment.

A REST API provides two core endpoints: `GET /live` returns the most recent measurement, while `GET /history` retrieves chronological data from a selected time range.

To focus on core infrastructure, the following elements are excluded from this MVP but are planned for future iterations:

- Graphical frontend dashboard for data visualization.
- Physical ESP32 hardware and real sensors.
- Automated user notifications (e.g., Push or Email alerts).

IV. TECHNICAL CONCEPT

This Section describes the overall technical architecture of the proposed system, including the embedded hardware, communication protocols, cloud infrastructure, data model, and planned extensions. The focus lies on the interaction of IoT devices with a scalable, serverless cloud backend for real-time and historical data processing.

A. Data Acquisition and Transmission

The local ESP32 microcontroller is equipped with multiple sensors for monitoring environmental conditions.

The DHT11 sensor measures temperature and humidity with a decent accuracy for indoor applications. This is complemented by the KY-028 thermistor for high precision temperature readings. The KY-026 flame sensor enables the detection of uncontrolled fire sources by measuring the intensity of infrared light emitted by flames [3]. Additional gas sensors (e.g., BME680 for volatile organic compounds, TSL2561 for light intensity) must be determined based on the specific measurements of interest.

The sensor data is read periodically, pre-processed on the ESP32, and encapsulated into a structured data object for transmission, as illustrated in Listing 1. Each message contains device metadata (ID, location, uptime) and a nested structure for sensor measurements for further extensibility.

```

1 {
2   "deviceId": "esp32-lab-01",
3   "runningSince": "10000",
4   "location": "Lab A",
5   "measurements": {
6     "dht11_temperature": {
7       "value": 20,
8       "unit": "degree C"
9     },
10    "dht11_humidity": {
11      "value": 50,
12      "unit": "%"
13    },
14    "ky026_flame": {
15      "value": 4000,
16      "unit": "analog"
17    },
18    "ky028_termistor": {
19      "value": 22,
20      "unit": "degree C"
21    }
22  }
23 }
```

Listing 1. Example of the JSON data model for sensor value encapsulation on the ESP32.

Messages are published to AWS IoT Core with the MQTT protocol and secured using TLS [4]. Certificate-based authentication ensures device integrity and prevents unauthorized publishing to the cloud backend.

B. Planned Technology Stack

The following programming languages and technologies are utilized across the different components of the system:

- C++: ESP32 firmware for sensor data acquisition and MQTT communication.
- Python: Implementation of AWS Lambda functions for data processing and validation.
- TypeScript: Infrastructure definition and backend logic within AWS services.
- React: Web-based frontend for visualization and interactive data analysis.

C. Cloud Processing Architecture

Incoming MQTT messages are processed within AWS using a serverless, event-driven architecture. The cloud backend is logically divided into two branches for data handling:

Live data is made available through a RESTful API endpoint `GET /live`, optimized for low-latency access to the most recent sensor values [5]. Historical data is persisted for long-term storage and analysis, accessible via `GET /history` for in-depth time-series evaluation and trend analysis.

Additionally, an alerting mechanism is planned to be implemented using AWS SNS to notify users via email when certain thresholds are exceeded, such as high temperature.

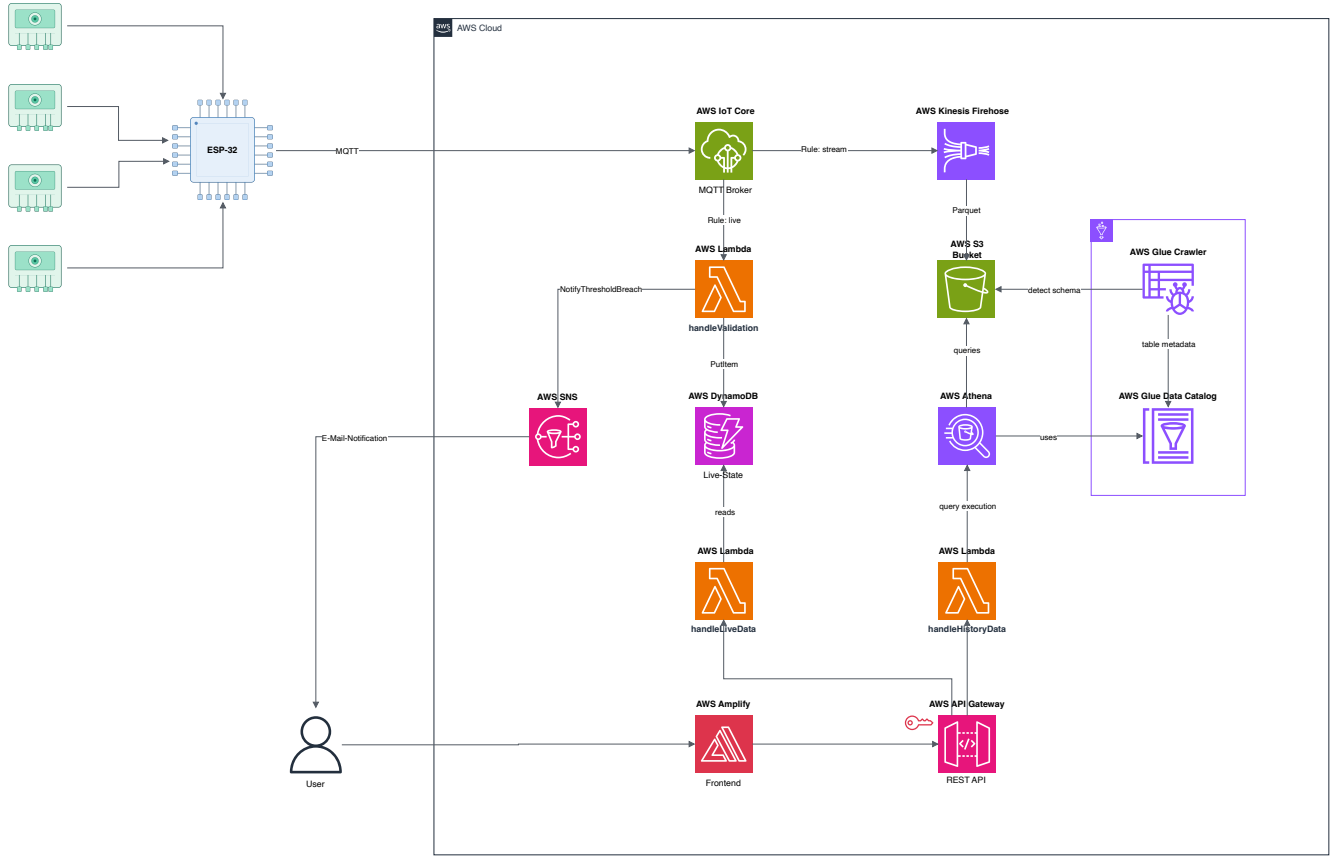


Figure 1. Cloud Processing Architecture

Future extensions may include the development of a React-based frontend dashboard for visualizing historical trends for specific time periods, enabling time-based filtering and comparison of environmental conditions, being hosted on AWS Amplify for seamless AWS service integration. Data security could be guaranteed with AWS Cognito for user authentication and authorization, ensuring secure access to the system and mitigating unauthorized access risks.

The overall architecture is represented in Figure 1, illustrating the flow of data from the ESP32 device through AWS to the API endpoints and potential alerting services.

D. Testability

The system's testability varies across services. Lambda functions are fully unit-testable via SAM CLI and support integration testing using DynamoDB Local or mocked dependencies. DynamoDB and S3 can be tested locally with DynamoDB Local and LocalStack respectively, while API Gateway supports end-to-end testing through SAM's local API emulation. Athena and Glue lack free local equivalents, requiring LocalStack Pro for integration tests. AWS IoT Core

cannot be tested locally at all, with Device Advisor being the only available validation tool. SNS supports partial local mocking but can be fully integration-tested via LocalStack.

E. Cost Estimation

The calculation of Table III is made to estimate the average operating cost per month for one device. Assumptions of 3 unique users, 8 hours per day and 22 days per month are made. As depicted, the difference between Free Tier and Post Free Tier costs is marginal.

V. CONCLUSION AND FUTURE WORK

Maintaining precise and reliable environmental conditions is a critical requirement for safety and quality assurance in modern laboratory facilities. This concept paper introduced Sensiq, a highly scalable, MCU-based IoT monitoring architecture designed to eliminate the limitations of isolated, local sensor networks. By leveraging a serverless AWS infrastructure—anchored by AWS IoT Core, Lambda, and DynamoDB—Sensiq ensures low-latency incident alerting, secure

Table III
COST ESTIMATION FOR AWS SERVICES

Service Details	Usage/Quantity	Free Tier	Post Free Tier
IoT Core [6]	518k msgs, 268k billed	~\$0.27	~\$0.27
Lambda – handleValidation [7]	518k inv, 128 MB / 200 ms	\$0.00	~\$0.06
Lambda – handleLiveData [7]	228k inv, 128 MB / 200 ms	\$0.00	~\$0.04
Lambda – handleHistoryData [7]	660 inv, 128 MB / 200 ms	\$0.00	~\$0.00
DynamoDB [8]	518k WCU, 228k RCU, storage	~\$0.01	~\$0.08
S3 (Parquet) [9]	~0.5 GB stored	~\$0.01	~\$0.02
Athena [10]	660 queries × 50 MB = 33 GB scanned	~\$0.16	~\$0.16
API Gateway [11]	229k requests	\$0.00	~\$0.08
SNS [12]	Minimal alert volume	\$0.00	~\$0.01
Total		~\$0.46	~\$0.73

end-to-end data transmission, and cost-effective historical data persistence without the overhead of managing dedicated servers.

The requirements established through persona-based user stories provide a clear development roadmap, prioritizing data sovereignty and real-time accessibility. For the Minimum Viable Product (MVP), the focus is placed on establishing a robust backend pipeline, replacing physical hardware with simulated telemetry to rigorously test data ingestion and REST API retrieval performance.

Future work will focus on the development and deployment of the requirements for the MVP. Subsequently, development will transition to integrating a physical ESP32 device equipped with the designated environmental sensors. Finally, the system will be extended with an interactive React-based frontend dashboard for comprehensive user-facing data visualization, alongside automated AWS SNS email notifications for instant incident response, thereby fulfilling the complete architectural vision of the Sensiq platform.

REFERENCES

- [1] Vergi Ramadhani, Syarifah Aini, and Jimmie. “Air Quality Monitoring System for Hazardous Gas Detection and Fire Risk in a Chemical Laboratory”. In: (2026). Accessed May 2026. URL: <https://ioinformatic.org/index.php/JAIEA/article/view/2109/1424>.
- [2] Geethanjali Gudduri. “Enhancing IoT Data-Stream Processing with a Lean Serverless Cloud Architecture”. Accessed May 2026. 2025. URL: <https://norma.ncirl.ie/9217/1/geethanjalgudduri.pdf>.
- [3] ArduinoModules. *Arduino Flame Sensor KY-026 — Wiring Diagram and Code*. <https://arduinomodels.info/ky-026-flame-sensor-module/>. Accessed: 2026-05-05. 2026.
- [4] Amazon. *Amazon Web Services*. [Online]. URL: <https://aws.amazon.com/>.
- [5] Roy Thomas Fielding. *Architectural styles and the design of network-based software architectures*. University of California, Irvine, 2000.
- [6] Amazon. *AWS IoT Core Documentation*. [Online]. URL: <https://docs.aws.amazon.com/iot/>.
- [7] Amazon. *AWS Lambda Documentation*. [Online]. URL: <https://docs.aws.amazon.com/lambda/>.
- [8] Amazon. *Amazon DynamoDB Documentation*. [Online]. URL: <https://docs.aws.amazon.com/dynamodb/>.
- [9] Amazon. *Amazon Simple Storage Service (S3) Documentation*. [Online]. URL: <https://docs.aws.amazon.com/s3/>.
- [10] Amazon. *Amazon Athena Documentation*. [Online]. URL: <https://docs.aws.amazon.com/athena/>.
- [11] Amazon. *Amazon API Gateway Documentation*. [Online]. URL: <https://docs.aws.amazon.com/apigateway/>.
- [12] Amazon. *Amazon Simple Notification Service (SNS) Documentation*. [Online]. URL: <https://docs.aws.amazon.com/sns/>.